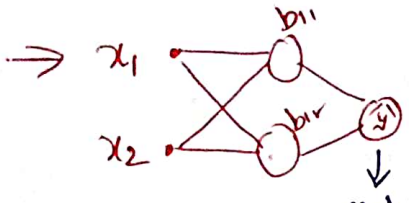
⊛ optimisers :- To improve the performance of NN & to speed the training ; No, 06, 15, 25

Techniques
- (i) weight Initialisation
- (ii) Batch Normalisation
- (iii) Activation func'
- (iv) optimisers ⊛⊛



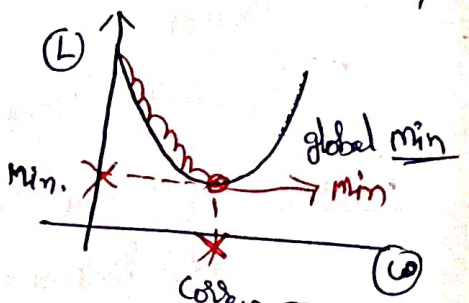
Consists of (9) learnable parameters, to get them is a challenging task;

⊛ Basically NN' is a optimization problem (i.e min' Loss)

⊛ In this process, we take random values for w, b then slowly we tries to iterations, we find req' w, b ;

⊛ Loss (or) loss func' graph $L(d_1, d_2, d_3 \ldots\ldots d_9)$ = (9) Dimensions Here ; means (9) directions ✓

⊛ Gradient Descent :- $W_n = W_0 - \eta\left(\frac{\partial L}{\partial w}\right)$



Types :-

① Batch GD :- at a single time, total data passed like 500 rows → at a time on 500 rows prediction cal'd & → loss → w_n weight updated ; like Subsets (or) even one datapoint @ a time rather than processing entire dataset

② stochastic GD :- say Epoch = 10 & 500 rows = 5000 times

③ mini Batch GD :- 1st batch size fixed;

say 5 batches → 10 × 5 times ✓

⊛ challenges in GD :-

① learning rate Fixing ; $W_n = W_0 - \eta\left(\frac{\partial L}{\partial w_0}\right)$ (LR)

η << Small = Causes slow convergence

ex :- η > large = Causes unstable training with big jumps ;

($\eta = 0.16$ ✓)

② learning rate Scheduling :- used to handle (LR) , pre defined for LR ;

③ No separate LR's for various weights ✓ :- (η) same for the both directions i.e (9) parameters ≠ (9) LR'

⊛ local Min. problem ✓

⑤ Saddle point :- one dimen' slope increases & another dimension slope dec' means (So) slope changes; that cases $\frac{\partial L}{\partial w} = 0$ ✓

- (GD) fails here;

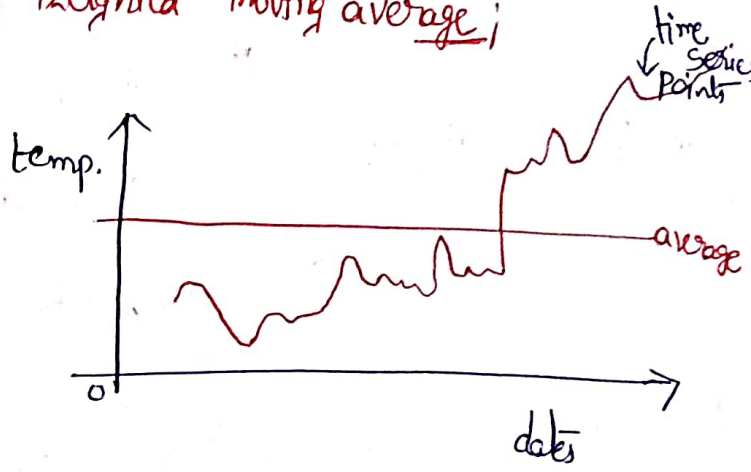
What next :- all are build with small changes in (GD) only ✓

① Momentum ② Adagrad ③ NAG ④ Rmsprop ⑤ Adam (⊛) ✓

In all common concept is - Exponentially weighted moving average;

⊛ EWMA :- take line series graph;

It is a Technique, using which we find ~~Redis~~ in time-series based data; trends



(Ex) :- stocks, Temp. data etc....

used in :- ① time Series ③ signal prosess
② financial ④ DL-optimizers ✓

- ~~It~~ is a method (statistical) used in DL to smooth out noisy data & capture underlying trends particularly in the context of optimization algorithm

- unlike a SMA, which treats all data points equally, EWMA gives more weight to recent observations & dimising weight to other ones; this helps capture more current trends, making it useful in various fields;

⊛ EWMA is highly effective Because :- ① Smoothing
② Responsiveness
③ Reducing Noise

(*) EWMA works :- To cal' EWMA, we assign a decay rate (or) smoothing factor (α) a value b/t 0 to 1 ; the smoothing factor controls how much weight is given to recent values Vs past value ;

$$(EWMA)_t = \alpha x_t + (1-\alpha) \times EWMA_{t-1}$$

(or)

$$S_t = \beta (S_{t-1}) + (1-\beta) x_t$$

x_t = current datapoint
α = Smoothing factor
$(EWMA)_{t-1}$ = prev. EWMA value ;

Ex :- 0.9, 0.7, 0.8, 0.6

Take $\beta = 0.8$

$S_1 = 0.8 \times 0.9 + 0.2 \times 0.9 = 0.9$
$S_2 = 0.8 \times 0.9 + 0.2 \times 0.7 = 0.86$
$S_3 = 0.8 \times 0.86 + 0.2 \times 0.8 = 0.848$
$S_4 = 0.8184$

(*) ↑ α = more weight given to recent data ;

(*) EWMA gives more weights to recent observations & less weight to older ones ;

Ex :- Daily stock price over 5 days :- Day 1 : 100
" 2 - 105
" 3 - 103
" 4 - 108
" 5 - 107

Normal ave = 104.6 ✓

Assume we start with an initial average ; let $\alpha = 0.5$ ✓ (50% of weight of latest data)

So (1) Day 1 : EWMA = 100 (Initial value)

2 :- $0.5 \times 105 + (1-0.5) \times 100 = 102.5$

3 :- $0.5 \times 103 + (1-0.5) \times 102.5 = 102.75$

4 :- 105.375

5 :- 106.1875

The result is a smoother trend line that emphasizes recent values without discarding past data ;

Key Takeaways :- (1) EWMA is a technique to capture trends by giving weight to recent points ;
(2) Smoothing factor
3) Its applications ;

⊛ Saddle point :- It is a Critical point in the optimization landscape of NN's loss func' where the (gradient = 0) but the point is neither a local min. (or) local max; characteristics :- Zero gradient, mixed curvature (slope too low, so training take so much time), Not an extremum;

⊛ this problem resolved with momentum optimizers;

⊛ Momentum optimization — the why? — To speed training of model;

— non-convex optimization because the loss function's curve is more complex

problems → Noisy gradient (local min.) not good;
→ high curvature
→ consistant gradient (slope is too slow)

— The above 3' issues will be solved with Momentum optimizer;

— What? :-

M = m × a V like in physics; it will behaves

— previous gradients all are moving (same direction);

— (Speed) than normal SGD, Batch GD;

Mathematical approach :-

— $W_{t+1} = W_t - \eta(\Delta w_t)$, In momentum, we consider (V) velocity @ time;

like $W_{t+1} = W_t - (V_t)$

$V_t = \beta \times V_{t-1} + \eta \Delta w_t$

current gradient by LR

momentum component

best $\beta = 0.9$ ✓

$0 < \beta < 1$

Momentum :- adds a memory of the previous gradients. It helps the optimizer to accelerate in the (Right) direction ✓
Reduces oscillations;

✱ $\alpha = \frac{1}{(1-\alpha)}$ days say $\alpha = 0.9$ means $= \frac{1}{1-0.9} = \frac{1}{0.1} = 10$ days ✓

indicates the last 10 days ka

i.e EWMA will act as a Normal average ✓ average hai;

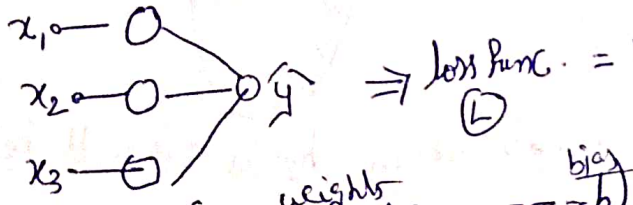
✱ For a same data points, if you varies $\beta(\alpha)$ α like 0.1, 0.5, 0.9, 0.98 then its intution ; if α is high, it means, you are giving high weightage to previous datapoints ; 10 days ✓

best $\alpha = 0.9$ ✓ actually they said β — Moody — Stable ✓

✱ $\alpha^3 < \alpha^2 \alpha \alpha$ if $0 < \alpha < 1$ ✓ $\alpha = 1 - \beta$ ✓

V100 Imp ✓

✱ SGD With Momentum :-



$\Rightarrow$ loss Func. $= y - \hat{y}$ (L) $\Rightarrow MSE = (y - \hat{y})^2$ ✓

$\Rightarrow L = f(w_1, w_2, w_3 \text{ (weights)}, b \text{ (bias)})$ say 5 weights, 3 bias

18 - Dimensions say diagram Build $\Rightarrow$

generally we see only 3D ✓;

only weight $\rightarrow$ 2D

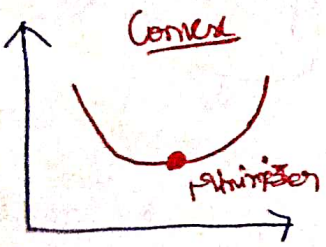
one (weight + bias) $\rightarrow$ 3D

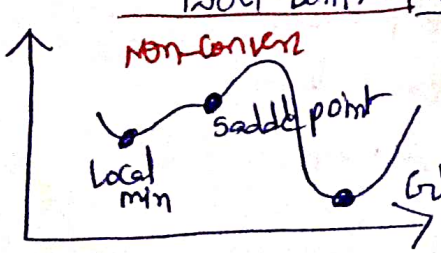
no. of weights, bias $\rightarrow$ 'n' D ✓

✱ Contour = 2D Representation;

✱ Google :- mesh() combines mesh/contour plot ;

✱ WolFram Alpha ✓



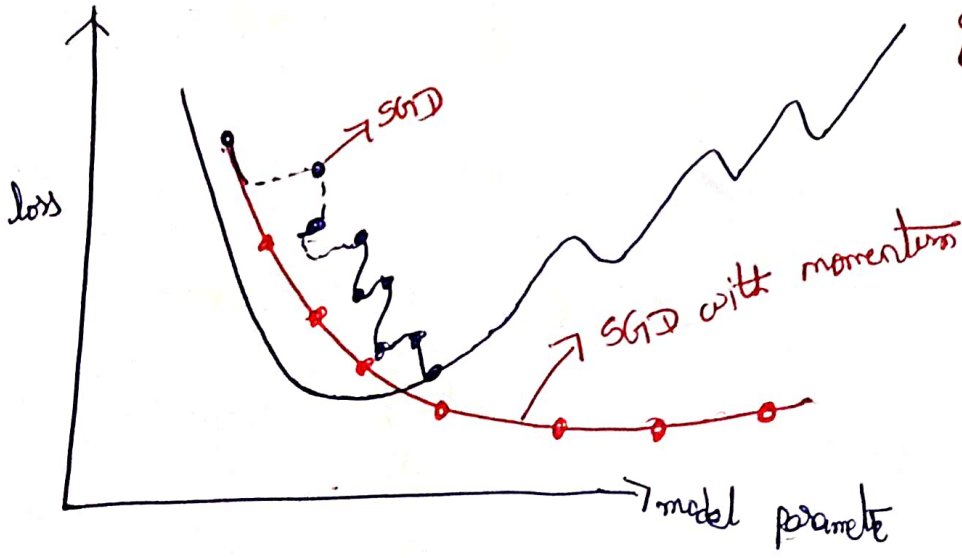


$\Rightarrow$ Finding minima is very difficult because ① Local min ② Saddle point ③ High curvature

Code of SGD with momentum :-

```
velocity = 0
gamma = 0.9  # momentum factor
learning rate = 0.1
for each epoch:
    grad = compute_gradient()
    velocity = gamma * velocity ⊕ learning_rate * grad
    weights = weights ⊖ velocity ✓
```

adv :- ⊛ faster Convergence , helps escape local minima , smooths noisy gradients



$$\theta = \theta - v_t$$

$$v_t = \gamma v_{t-1} + \eta \Delta L(\theta)$$

θ = weights

$\nabla L(\theta)$ = gradient of loss fun' with respect to θ

drawbacks :-

* Potential for overshooting the minimum due to accumulated momentum;
* delayed updates when encountering sharp turns in the loss landscape;
* sensitivity to hyper parameters like β, η;

✱ Nesterov Accelerated Gradient (NAG) :-

Momentum mathematics :- $W_{t+1} = W_t - V_t$ where $V_t = \beta V_{t-1} + \eta \nabla W_t$

V_{t-1} indicates past velocity i.e history of velocity ①

∇W_t gradient @ that point

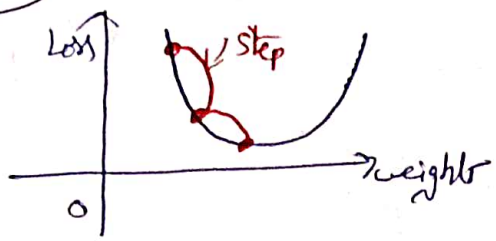
β = decay factor (or) moment coeff

If $\beta = 0$ then it becomes SGD

$\beta = 1$ not good i.e

generally $\beta = 0.9$ ✓

because of βV_{t-1} term the steps will be big like



-where as NAG :- better version of previous case;

look ahead = ఎదురుచూసి చూడటం

" = plan for upcoming events

$$W_{la} = W_{lookahead} = W_t - \beta V_{t-1}$$

$$V_t = \beta V_{t-1} + \eta \nabla W_{la} \quad (or) \quad V_t = (W_t - W_{la}) + \eta \nabla W_{la}$$

$$W_{t+1} = W_t - V_t \checkmark$$

✱ In SGD momentum case, we apply both terms history of velocity & gradient @ that point but where in NAG, finding look ahead point before computing the gradient.

✱ this small change gives better accuracy & faster convergence, especially for tricky loss landscapes.

Why NAG is better :- → might overshoot minima (too much speed)

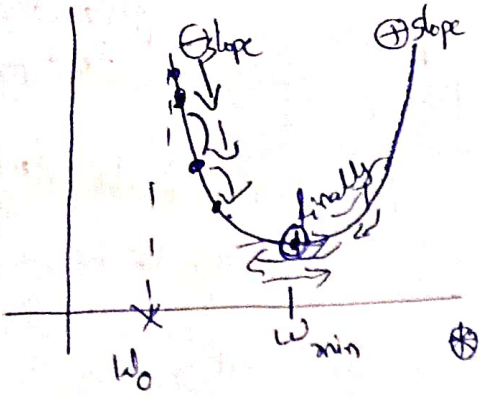
momentum — Go where the hill pushes me, with some memory of how fast I was going.

NAG — look ahead where I would land, check if the direction is still good, then adjust my moment. Its like applying brakes before a sharp turn — you're more cautious & accurate;

gently rolls into minima with foresight

NAG = Momentum ⊕ "look ahead" gradient

Geometric Intuition :-

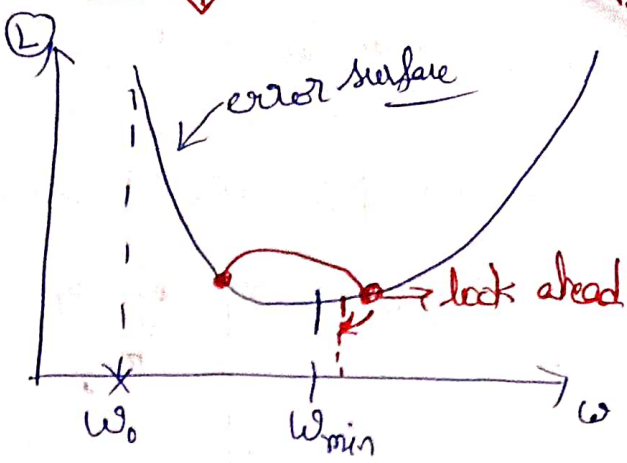


momentum case ↓

@ every point we cal. both momentum & gradient.

⊛ -ve of gradient;

NAG ↓



look ahead cal' with momentum & then only gradient cal' but momentum term;

Keras Code ↓

```
tf.keras.optimizers.SGD(
    learning_rate=0.01, momentum=0.0, nesterov=False, name="SGD", **kwargs
)
```

For only SGD means ⇒ momentum = 0, nesterov = False;
SGD+momentum ⇒ " = 0.9, " "
NAG ⇒ " = " , nesterov = True

disadv :- ① more computation per step
② Tricky to tune (η, β)

Simply NAG :-

Imagine you're riding a bicycle down a hill, & you want to go faster,
— Instead of just pushing the pedals immediately, you first (lean forward) to check if the road ahead at ⓒ has a curve; Then (based on what you see), you decide how much to pedal & which way to steer;

In terms of NAG :-

— Before making the next move;
— you (look ahead using momentum) to guess where you're heading;
— then, you cal' the gradient @ the (future spot) instead of current spot;
— this helps you make a smarter & more accurate update;

peeking ahead = checking the effect of your momentum (before) blindly following it

Adagrad :- Adaptive Gradient is an optimization algorithm that adapts the (η) for each parameter individually, based on how frequently it's been updated during training ;

Intuition :-

Imagine you're learning to play multiple instruments :

- you practice guitar a lot (& you're learned fast);
- But you rarely practice piano (still a beginner)

so Adagrad gives smaller steps (learning rates) to things which you're already practiced a lot & bigger steps (η) to what you're barely touched (piano);

So in DL :-

{ Frequently updated weights ⟶ get smaller learning rates;
{ Rarely updated weights ⟶ get larger learning rates ✓

this is especially useful for (sparse data) (like NLP & recomm. system)

↳ means 0(os) 1

update Rule :- $$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t \oplus \epsilon}} \cdot g_t$$

where $G_t = \sum_{\gamma=1}^{n} g_\gamma^2$

ϵ = small number to avoid division by zero

θ = model parameters
η = LR
G_t = Sum of squares of past gradients
g_t = gradient @ time step (t)

this means the (η) automatically dec' over time for each parameter.

pro's :- (1) Adaptive (η) → each parameter has its own (η) (2) Good for sparse data → NLP's (or) one-hot encoded data

(3) No manual η decay

(4) simple to implement → easy to code & understand

con's :-

(1) Poor for complex models with dense gradient

(2) No mechanism to "restart"

(3) η shrinks too much → eventually becomes too small & stops learning ✓

summary

	optimisers	(η)	feature
(1)	SGD	fixed	small but inefficient sometimes
(2)	momentum	fixed + memory	uses past gradients too
(3)	Adagrad	Adaptive	Auto-adjusts for each (w) (β)

→ the Adagrad optimiser automatically adjusts (η) per parameter;

RMSprop :- Root mean square propagation; — improved version of (Adagrad);

— It's an adaptive learning rate optimiser that tries to solve Adagrad's biggest prob.

— i.e In adagrad, the (η) keeps shrinking over the time until it becomes too small to learn anything new;

— RMSprop fixes this by, it keeps a moving average of recent squared gradients & uses that to adapt the (η);

① update moving average of squared gradients :-

$$E(g^2)_t = \beta E(g^2_{t-1}) + (1-\beta)\, g_t^2$$

② update parameters :- (Normalize by η)

$$\theta_{t+1} = \theta_t - \frac{\eta \cdot g_t}{\sqrt{E(g^2)_t + \epsilon}}$$

θ_t : model parameters

$\nabla w_t = g_t$ = gradient @ time t

η = LR

β = decay rate (0.9)

ϵ = small number for stability

Adv :-

① solve adagrad's "too small η" problem;

② Works well for non-stationary objects;

③ often better than plain SGD for RNN's deep CNN's

④ Faster convergence

Con's :-

⊛ Not always the fastest on all datasets;

② Don't use momentum by default; but RMSprop + momentum ✓

```
model.compile( optimizer = tf.keras.optimizers.RMSprop(η = 0.01, rho=0.9), loss='mse'
```

⊛ Rmsprop builds on the limitations of standard GD by adjusting the ② dynamically for each parameter. It maintains MA of squared gradients to normalize the update, preventing the drastic ② fluctuations;

Rmsprop algorithm :-

Start : Initialize Parameters

↓

Compute gradient g_t

↓

Update squared Gradient - Moving ave.

↓

Update parameters (normalize)

↓

Check convergence

↓

Stop if converged (or) else Repeat ↺

⊛ Adam optimiser :- Adaptive moments Estimation is like a combo meal of:

① momentum - keeps track of running average of past gradients (1st moment) so you don't zig-zag ✓

② Rmsprop - keeps track of a running average of squared gradient (2nd moment) so each parameter gets its own adaptive learning rate;

Key points :-

① Hybrid approach :- momentum (1st) = m_t ⊕ Rmsprop (2nd) = v_t adds benefits of both speed & stability

② Adaptive learning rates - Each parameter has its own adjusted (?) based on gradient history

③ Bias correction :- Adam applies bias correction for m_t & v_t

④ Fewer hyperparameter tuning headaches :- Works well with default settings :-

$\eta = 0.001$ $\quad \beta_2 = 0.999$

$\beta_1 = 0.9$ $\quad \epsilon = 10^{-8}$ ✓

⑤ Good for sparse gradients :- like NLP, recommendation systems;

⑥ Fast convergence;

⑦ memory req' :- stores 2-extra buffers/parameter (m_t & v_t) so memory usage is higher than vanilla SGD

⑧ Drawback :-

Can overfit (&) fail to convergence to the best solution in some cases;

⑨ Variants :- AdamW = better weight decay

Nadam = Adam + Nesterov momentum

AMSGrad - modified to prevent a rare convergence issue.

Mathematical steps :-

① Update biased moment estimates :-

$$1^{st} \text{ moment} - m_t = \beta_1 m_{t-1} \oplus (1-\beta_1) g_t$$

$$2^{nd} \text{ moment} - v_t = \beta_2 v_{t-1} \oplus (1-\beta_2) g_t^2$$

② Bias correction :- (to fix early step bias towards zero) :-

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \quad \& \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

③ parameter update :-

$$\theta_t = \theta_{t-1} - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Intuition :- Adam is like a driver who :

- uses past velocity (momentum) to keep moving in right direction ;
- uses the road's slope info (RMSprop) to avoid jerking around & overshooting

Regularization :- (R)

structured pattern of activities that an individual consistently

— Regularization in DL is like giving your NN a "discipline routine" so it doesn't overfit the training data;

— without it, the network might memorize the every tiny detail from the training set, which makes it bad @ generalizing to new, unseen data;

— (R) Techniques work by add constraints (or) penalties to model, encouraging simpler & more generalizable solutions;

✱ Why (R) is needed :- ① overfitting
② Too many parameters

✱ Types :- L_1 Regularization — Lasso
L_2 Regularization — Ridge
} Both add a penalty term to the loss func' to discourage large weights
Dropout —
early stopping —
Batch Normalization —
Data Augmentation;

L_1 Regularization (Lasso) :-

(Lasso = least absolute shrinkage & selection operator)

① Adds the sum of absolute weights as a penalty term to loss func'

$$loss_{total} = loss_{original} + \lambda \left(\sum_i^n |w_i| \right)$$

— effect : encourages sparsity i.e some weights becomes exactly zero;
— useful for feature selection;

λ = which determines the trade off b/t (bias & variance) in coeff,

* Although, it's used to resolve overfitting, (L_1) regularization can actually cause model (underfitting) based on the lambda value;

* if ($\lambda = 0$) then (R) is disabled;

Concept of :- $MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - y_{predicted})^2$

(y_i → Truth)

Say $y_{predicted} = h(\theta)$ = higher order PL $= \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \cdots$

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - h_\theta(x_i))^2$$

(L_2) regularization :→

Now I want to minimize this error (MSE) like by introducing $\lambda \sum_{i=1}^{n} \theta_i^2$ (or) $\lambda \sum_{i=1}^{n} w_i^2$

* adds the sum of square weights to the loss func'
* Effect: shrinks weights towards zero;
* prevents weights from growing too large, smoothes the model; if your (θ) gets bigger then $\lambda \sum_{i=1}^{n} \theta_i^2$ will big ∴ finally error will big

to avoid this we (add penalty) to θ's

L_1 Regularization :- $MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - h_\theta(x_i))^2 + \lambda \sum_{i=1}^{n} |\theta_i|$

* google :- sklearn lasso regression ✓
  ridge " ✓

Elastic Net :- $(L_1 + L_2)$ ✓